

Prácticas Tema 4

Nombre de la actividad: Primera práctica Tema 4

Contenido relacionado: Tema 4 Clases

Los sistemas operativos multitarea actuales permiten la ejecución concurrente (al mismo tiempo) de varios programas a la vez. Para realizarlo, el núcleo del sistema operativo dispone de una entidad denominada planificador que, de acuerdo a una serie de características, selecciona un programa como el próximo a ser ejecutado.

Todos los programas que pueden ser ejecutados en un sistema operativo disponen de las siguientes características:

- Un identificador de proceso (pid) de tipo integer que permite distinguir de forma unívoca un programa de otro.
- Una prioridad en la que se basará el planificador para determinar cuál es el próximo programa a ejecutar. Podrá tomar los valores 4, 3, 2, 1 (siendo 4 la prioridad más alta y 1 la más baja), podemos utilizar la prioridad 0 para expresar la prioridad del proceso nulo.
- Tienen la capacidad de poder mostrar en la consola la información sobre su identificador, nombre y prioridad.

La clase para representar estos objetos se llamará `Process` y contiene los atributos indicados, métodos de acceso a dichos atributos, diferentes constructores para instanciar :

- Constructor indicando su nombre (en este caso la prioridad del proceso será 2)
- Constructor indicando su nombre y prioridad.
- El identificador del proceso (pid) vendrá dado por el número secuencial del proceso instanciado.

, y el método `toString()` .

Además, existen tipos de programas/procesos especiales, uno de ellos denominado de tiempo real, que siempre tienen una prioridad muy alta (valor 4) , unos programas que se ejecutan en segundo plano que tienen la prioridad más baja (1), finalmente el proceso nulo tiene la prioridad más baja posible, 0. Además, estos programas especializan el comportamiento de mostrar por consola la información de identificador, nombre y prioridad, anteponiendo el tipo de objeto que son (de tiempo real o de segundo plano).

Apartado 1. Jerarquía de clases

Se pide programar el conjunto de clases

(**Process**, **RealTimeProgram**, **BackGroundProgram**) que definen el comportamiento de los programas según las características descritas anteriormente.

Apartado 2. Planificador

Se quiere definir una clase **Scheduler** (planificador) que sea capaz de gestionar programas con prioridad. Esta clase tendrá un atributo que será un array, o una lista, de **Process**, donde se almacenarán los programas a gestionar. El constructor de la clase recibe como parámetro el número máximo de programas que el planificador es capaz de gestionar.

Se pide programar un método que sea capaz de almacenar programas ordenados de menor a mayor prioridad: `void add(Process process).`

Apartado 3. Método `next()`

Programar un método en la clase planificador que devuelva el próximo programa que debe ejecutarse (el de mayor prioridad): `Process next().`

Nota: Se deja a elección del alumno, mediante la programación de la ordenación, decidir ante niveles de prioridad iguales qué programa devolver.

Método main en la clase Scheduler

```
public static void main(String[] args) {
    Process programa1 = new Process("programaNormal", 2);
    Process programa2 = new RealTimeProgram("tiempoReal");
    Process programa3 = new BackGroundProgram("segundoPlano");
    Scheduler planificador = new Scheduler(3);
    planificador.add(programa1);
    planificador.add(programa2);
    planificador.add(programa3);
    System.out.println(planificador.next());
    System.out.println(planificador.next());
    planificador.add(programa3);

    System.out.println(planificador.next());
    planificador.add(programa2);
    System.out.println(planificador.next());
    System.out.println(planificador.next());
    //Se devuelve el proceso nulo en caso de no tener procesos
    System.out.println(planificador.next()); } }
```

Resultados esperados:

```
RealTimeProgram Process [pid=2, priority=4, name=tiempoReal]
Process [pid=1, priority=2, name=programaNormal]
BackGroundProgram Process [pid=3, priority=1, name=segundoPlano]
RealTimeProgram Process [pid=2, priority=4, name=tiempoReal]
BackGroundProgram Process [pid=3, priority=1, name=segundoPlano]
Proceso nulo
```

Fecha de límite de entrega: Una semana desde el día de trabajo en clase

Formato y espacio de entrega:

- La entrega se realizará a través de Blackboard.
- Se entregará en formato digital en un archivo comprimido en el enlace T4_Entrega1 del módulo de aprendizaje del Tema 4. **Se espera un documento pdf o en su defecto doc con el código fuente y el resultado de la consola.**
- Si es demasiado pesado para subir directamente a Blackboard, subirlo a vuestra nube de OneDrive de u-tad y compartido a través de Blackboard con la opción de subir archivo de almacenamiento en la nube.
- El alumno recibe un mail de confirmación cada vez que hace una entrega. Es su responsabilidad revisar que la actividad se ve correctamente (en caso de no visualizarla, contactar con el profesor).
- Sólo se permite una entrega.
- No se permitirán entregas fuera de plazo.